

Chapter 6

Algorithm Chains & Pipelines

Chaining preprocessing and models
without leaking data

Learning Objectives

- See why preprocessing **before** cross-validation leaks information
- Build a `Pipeline` that chains steps into one estimator
- Grid-search over preprocessing **and** model parameters together
- Use `make_pipeline` and inspect steps via `named_steps`
- Even grid-search **which model** to use

Why Chain Steps?

Real ML is rarely a single algorithm — it's a **sequence** of steps.

- Scale → extract features → select features → model
- Example: a kernel SVM on `cancer` jumps to **0.95** after `MinMaxScaler`

```
scaler = MinMaxScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
svm.fit(X_train_scaled, y_train)          # test score 0.95
```

Doing this by hand is error-prone — especially inside cross-validation.

The Leakage Problem

A **naive** grid search over an SVM using **pre-scaled** data:

```
scaler = MinMaxScaler().fit(X_train)          # fit on ALL training data
X_train_scaled = scaler.transform(X_train)
GridSearchCV(SVC(), param_grid, cv=5).fit(X_train_scaled, y_train)
```

- The scaler already saw the **whole** training set — including each CV **validation fold**
- Those folds are supposed to stand in for *unseen* data
- Result: **over-optimistic** CV scores and possibly **suboptimal** parameters

Preprocessing Must Be Inside CV

The cross-validation split must happen **before** any preprocessing.

- Any step that **learns from data** (a scaler, feature selection) must be fit on the **training folds only**
- Cross-validation should be the **outermost loop** of your process

Solution: wrap the steps in a *Pipeline* so the whole chain is treated as one model — and re-fit inside every fold.

The Solution: Pipeline

Pipeline glues steps into a single estimator with `fit` / `predict` / `score`.

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([("scaler", MinMaxScaler()),
                  ("svm", SVC())])
```

- Each step is a **(name, estimator)** tuple
- All steps except the last must have a `transform` method
- Names are yours to choose (just no double underscore `__`)

How fit / predict / score Flow

```
pipe.fit(X_train, y_train)      # scaler.fit_transform → svm.fit
pipe.score(X_test, y_test)     # scaler.transform → svm.score → 0.95
```

- **fit**: calls `fit_transform` on each step in turn, feeding output to the next; **fit** on the last
- **predict / score**: `transform` through the steps, then predict on the last
- Same result as manual scaling — but far less code and no bookkeeping

Pipelines in Cross-Validation

Now the scaler is re-fit **within** each fold — correctly.

```
from sklearn.model_selection import cross_val_score
cross_val_score(pipe, X, y, cv=5)      # scaler.fit on training folds only
```

The whole chain is one model → each fold's validation data stays truly unseen.

Using Pipelines in Grid Search

Reference a step's parameter as `stepname__param` (double underscore).

```
param_grid = {'svm__C': [0.001, 0.01, 0.1, 1, 10, 100],
              'svm__gamma': [0.001, 0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(pipe, param_grid, cv=5)
grid.fit(X_train, y_train)
grid.score(X_test, y_test)      # 0.97 (MinMaxScaler refit per fold – no leak)
```

Compare to the naive version: same code shape, but now **leak-free**.

! Illustrating Information Leakage

100 samples, **10,000 random** features, a **random** target — nothing to learn.

```
select = SelectPercentile(percentile=5).fit(X, y)    # select outside CV
cross_val_score(Ridge(), select.transform(X), y, cv=5).mean()    #  $R^2 = 0.91$  (!)
```

- Selecting features on **all** data finds ones correlated with the target **by chance** → bogus 0.91
- Inside a pipeline (selection per fold): **$R^2 = -0.25$** — correctly says "no signal"

Feature-selection leakage is **far worse** than scaling leakage.

Part 2

The General Pipeline Interface

The General Interface

A pipeline can chain **any number** of estimators — feature extraction, selection, scaling, then a model.

- Every step **except the last** must have a `transform` method
- The **last step** needs only a `fit` method
- The last step can be a classifier, **regressor**, or even a transformer (e.g. PCA)

```
pipe = Pipeline([("scaler", StandardScaler()),  
                 ("pca", PCA()), ("ridge", Ridge())])
```

make_pipeline

A convenience that **auto-names** each step after its class.

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(MinMaxScaler(), SVC(C=100))
pipe.steps          # names: 'minmaxscaler', 'svc'
```

- Names are the lowercased class name
- Duplicate classes get suffixes: `standardscaler-1`, `standardscaler-2`
- For clarity with repeats, prefer explicit `Pipeline([...])` names

Accessing Step Attributes

Reach inside a fitted pipeline with `named_steps`.

```
pipe.fit(cancer.data)
components = pipe.named_steps["pca"].components_
components.shape      # (2, 30)
```

`named_steps` is a dict from step name → the fitted estimator.

Attributes in a Grid-Searched Pipeline

After a grid search, the best chain lives in `best_estimator_`.

```
pipe = make_pipeline(StandardScaler(), LogisticRegression())
grid = GridSearchCV(pipe, {'logisticregression__C': [0.01, 0.1, 1, 10, 100]},
                    cv=5).fit(X_train, y_train)

grid.best_estimator_.named_steps["logisticregression"].coef_
```

A common reason to use pipelines: inspect the tuned model's coefficients or importances.

Part 3

Grid-Searching the Whole Workflow

Grid-Searching Preprocessing + Model

Tune preprocessing parameters **jointly** with the model — even the polynomial degree.

```
pipe = make_pipeline(StandardScaler(), PolynomialFeatures(), Ridge())
param_grid = {'polynomialfeatures__degree': [1, 2, 3],
              'ridge__alpha': [0.001, 0.01, 0.1, 1, 10, 100]}
GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1).fit(X_train, y_train)
```

On Boston: best is **degree 2, alpha 10** → **0.77**; without polynomials only **0.63**.

The data *chooses* whether interactions help.

Grid-Searching Which Model To Use

You can even search over the **estimator itself** — pass a list of dicts.

```
pipe = Pipeline([('preprocessing', StandardScaler()), ('classifier', SVC())])
param_grid = [
    {'classifier': [SVC()], 'preprocessing': [StandardScaler(), None],
     'classifier__C': [0.1, 1, 10], 'classifier__gamma': [0.01, 0.1, 1]},
    {'classifier': [RandomForestClassifier(n_estimators=100)],
     'preprocessing': [None], 'classifier__max_features': [1, 2, 3]},
]
```

Set a step to **None** to skip it (RF needs no scaling).

On cancer: **SVC + StandardScaler, C=10, gamma=0.01 → 0.98** test.

A Caution on Search Size

`GridSearchCV` tries **every combination** of the specified parameters.

- Each extra parameter multiplies the number of model fits → **exponential** growth
- Searching preprocessing + model + model-choice is powerful but expensive
- Keep grids **coarse** first, refine around promising regions
- Only include steps whose value you actually want to test

Chapter Summary

- Preprocessing **outside** cross-validation **leaks** test information into training
- `Pipeline` chains steps and applies them correctly inside CV and grid search
- Reference step parameters with `stepname__param`
- `make_pipeline` auto-names steps; `named_steps` inspects them
- You can grid-search **preprocessing, model, and even model choice** together
- Watch the combinatorial cost of large grids

Next: Chapter 7 — Working with Text Data

End of Chapter 6

Pipelines make correct, reusable ML workflows